# Hazard Analysis
# FPVR

Team 9, Software Mandems
John Cho
Ibrahim Tariq
Vikram Chandar
Sankalpa Chhetri Dhakal
Arjun Sasidaran

Table 1: Revision History

| **Date** | **Developer(s)** | **Change** |
|---|---|---|
| Sept 30 | Sankalpa Chhetri Dhakal | Completed Introduction |
| Oct 6 | Sankalpa Chhetri Dhakal | Completed Scope and Purpose and System Boundaries |
| ... | ... | ... |

# Contents


**1 Introduction** **1**

**2 Scope and Purpose of Hazard Analysis** **1**

**3 System Boundaries and Components** **1**

3.1 VR Headset . . . . . 1

3.2 Controller Thumbsticks, Triggers, Buttons, and Input Connection 1

3.3 Host Computer . . . . . 2

3.4 Unity Application . . . . . 2

3.5 Drone Physics Simulation . . . . . 2

3.6 VR Rendering and XR Subsystem . . . . . 3

3.7 Training and Scoring System . . . . . 3

3.8 User Interface . . . . . 3

3.9 Data Storage . . . . . 3

**4 System Boundaries and Components** **4**

**5 Critical Assumptions** **4**

**6 Failure Mode and Effect Analysis** **5**

**7 Safety and Security Requirements** **5**

7.1 Functional Requirements . . . . . 5

**8 Non-Functional Requirements** **9**

**9 Roadmap** **9**

**10 Appendix — Generative AI Use Disclosure** **10**

10.1 AI Tools Used . . . . . 10

10.2 How and Where Used . . . . . 10

10.3 Verification of AI-Generated Content . . . . . 10

**11 Appendix — Reflection** **11**

[You are free to modify this template. —SS]

# 1 Introduction

[You can include your definition of what a hazard is here. —SS]

# 2 Scope and Purpose of Hazard Analysis

# 3 System Boundaries and Components

The system boundary includes the software and hardware directly involved in running and interacting with the FPV drone training simulator. This includes the VR headset, input devices, host computer, Unity application, drone simulation, training functionality, user interface, and locally stored data. The user and their physical surroundings interact with the system and are considered when identifying hazards, but they are not components of the software system itself. Real FPV drones and real-world drone operation are outside the system boundary.

The system consists of several major software and hardware components. These components are considered independently during hazard identification because a failure in one component may affect the safety or correctness of the overall system.

## 3.1 VR Headset

The VR headset, expected to be a **Meta Quest 3S**, provides the user's visual representation of the simulated environment and tracks the position and orientation of the user's head.

Potential hazards associated with this component include loss of tracking, visual latency, incorrect orientation, poor headset fit, and the user becoming unaware of their physical surroundings.

## 3.2 Controller Thumbsticks, Triggers, Buttons, and Input Connection

Controller thumbsticks and triggers provide axis values that are mapped to drone throttle, pitch, roll, and yaw. Buttons provide discrete commands such

as pause, restart, and respawn. Depending on the final implementation, these controls may be provided by VR controllers, a game controller, or an external FPV-style radio controller. The controller connection carries these values and button events to the application.

Failures may include thumbstick drift, where a centred stick reports an input and causes unintended drone movement; a stuck thumbstick or trigger, which maintains a control value after release; or a stuck or unresponsive button, which repeats or prevents a command. Incorrect axis mappings, inverted axis directions, or incorrect centre, range, or dead-zone calibration may produce unintended throttle or steering responses. A disconnected controller, dropped input updates, or delayed input delivery may cause loss of control, continued use of a stale control value, or a delayed response to a pilot command.

## 3.3 Host Computer

A computer executes the Unity application and associated simulation software.

Failures may include insufficient hardware performance, overheating, loss of power, driver issues, or resource exhaustion.

## 3.4 Unity Application

Unity provides the primary execution environment for the game.

This component is responsible for scene management, game logic, user interaction, rendering, and communication between major software modules.

## 3.5 Drone Physics Simulation

The drone physics subsystem models properties such as:

- thrust;
- gravity;
- acceleration;
- angular velocity;
- momentum;
- collisions; and
- drone response to pilot input.

Incorrect behaviour in this component represents an important hazard because users are expected to learn flying behaviour from the simulator.

### 3.6 VR Rendering and XR Subsystem

This subsystem renders the virtual environment and communicates with the VR headset.

Failures may result in visual artifacts, low frame rate, latency, incorrect camera movement, or loss of tracking.

### 3.7 Training and Scoring System

The training subsystem may contain tutorials, objectives, checkpoints, scoring metrics, crash detection, and user performance feedback.

Incorrect scoring or feedback could indicate that unsafe or incorrect flying behaviour is acceptable.

### 3.8 User Interface

The interface includes menus, settings, warnings, calibration options, pause controls, and training instructions.

Poor interface design could prevent the user from understanding warnings or exiting the simulation quickly.

### 3.9 Data Storage

The system may locally store information such as:

- user settings;
- controller configuration;
- training progress;
- scores; and
- application configuration.

Data corruption or unauthorized modification could affect system operation or training results.

# 4 System Boundaries and Components

[Dividing the system into components will help you brainstorm the hazards. You shouldn't do a full design of the components, just get a feel for the major ones. For projects that involve hardware, the components will typically include each individual piece of hardware. If your software will have a database, or an important library, these are also potential components. —SS]

# 5 Critical Assumptions

1. **Stable Internet Connection:** Users have access to a sufficiently stable internet connection for online features, including forms, multiplayer elements, and reviews.
   **Rationale:** Online features require reliable network connectivity to transmit user input, synchronize multiplayer gameplay, and submit or retrieve reviews. This assumption allows the system to be designed around the availability of network connectivity.

2. **Honest User Feedback:** Users provide honest gameplay reviews and feedback, and accurately report their experiences with the game.
   **Rationale:** Honest feedback helps ensure that reviews and user evaluations accurately represent gameplay experiences, allowing developers to identify issues and measure goals accurately without accounting for fake comments.

3. **Available Storage Capacity:** Users have sufficient available storage space on their systems to install the game and store any locally saved gameplay data.
   **Rationale:** The game requires storage for its installation files, course assets, and locally saved data. Insufficient storage may prevent installation, updates, or the saving of gameplay progress. It may require additional storage after progressing through levels such as save states and leaderboards, this storage should be available for the simulation to use.

4. **Basic Gaming Familiarity:** Users have basic familiarity with video games and standard user interface elements, including start screens, settings menus, and navigation controls.
   **Rationale:** This familiarity allows users to navigate the application and access gameplay features without requiring extensive introductory instruction, reducing the complexity of the user interface design.

5. **Suitable Hardware Performance:** Users have VR equipment that meets the minimum hardware requirements for running the game.
   **Rationale:** Meeting the minimum hardware requirements helps ensure that the system can maintain acceptable performance, provide a responsive VR experience, and execute the simulation as intended.

6. **Accurate Self-Reported Drone Experience:** Users who identify themselves as competent or experienced drone operators accurately represent their level of drone operating experience.
   **Rationale:** The system may rely on users' self-reported experience to understand their familiarity with drone operation and interpret their training results. Assuming these claims are accurate helps ensure that user classifications and performance evaluations are based on their stated level of experience.

# 6 Failure Mode and Effect Analysis

[Include your FMEA table here. This is the most important part of this document. —SS] [The safety requirements in the table do not have to have the prefix SR. The most important thing is to show traceability to your SRS. You might trace to requirements you have already written, or you might need to add new requirements. —SS] [If no safety requirement can be devised, other mitigation strategies can be entered in the table, including strategies involving providing additional documentation, and/or test cases. —SS]

# 7 Safety and Security Requirements

[Newly discovered requirements. These should also be added to the SRS. (A rationale design process how and why to fake it.) —SS]

`‘latex

## 7.1 Functional Requirements

**SR1:** The system shall provide users with a safety notice before their first VR session, reminding them to clear their physical play area of obstacles and remain aware of their surroundings.
**Rationale:** Users may collide with nearby objects while immersed in the virtual environment, potentially causing injury or damaging equipment.
**Risk:** Medium

**SR2:** The system shall allow users to pause or exit gameplay at any time during a VR session.
**Rationale:** Users may experience motion sickness, dizziness, or discomfort that requires them to stop playing immediately.
**Risk:** High

**SR3:** The system shall suspend active gameplay when the VR platform reports a headset disconnection or loss of tracking that prevents reliable interaction.
**Rationale:** Continuing gameplay when tracking is unreliable may cause unexpected interactions and increase user disorientation.
**Risk:** High

**SR4:** The system shall verify that required VR hardware is detected and initialized before allowing users to begin a flight session.
**Rationale:** Missing or improperly initialized hardware may prevent reliable controller input and compromise gameplay.
**Risk:** Medium

**SR5:** The system shall validate controller input values before applying them to the drone simulation and reject values outside supported input ranges.
**Rationale:** Invalid controller inputs may produce unintended drone movements or unpredictable simulation behaviour.
**Risk:** Medium

**SR6:** The system shall verify connectivity to required external services before enabling features that depend on them.
**Rationale:** Service connectivity failures may prevent multiplayer sessions or cause dependent features to behave incorrectly.
**Risk:** Medium

**SR9:** The system shall validate data received from multiplayer clients before accepting it for gameplay state updates or performance record storage.
**Rationale:** Malformed or manipulated network messages may cause inconsistent multiplayer behaviour or compromise recorded statistics.
**Risk:** High

**SR11:** The system shall notify users when a network interruption prevents access to multiplayer functionality.
**Rationale:** Users may otherwise be unaware that multiplayer functionality is unavailable or their session has been interrupted.
**Risk:** Low

**SR12:** The system shall disclose what gameplay data is collected, how it is used, and whether it is transmitted to external services before collecting user-associated data.
**Rationale:** Users need to understand how their gameplay information is handled to make informed decisions about their privacy.
**Risk:** Medium

**SR13:** The system shall require users to authenticate before accessing personal accounts or user-specific gameplay records, if account functionality is implemented.
**Rationale:** Authentication helps prevent unauthorized users from accessing personal information and saved progress.
**Risk:** High

**SR14:** The system shall enforce authorization checks before allowing users to view, modify, or delete stored gameplay records.
**Rationale:** Authorization prevents users from accessing or modifying records belonging to other users.
**Risk:** High

**SR16:** The system shall validate and sanitize user-provided data before processing or storing it.
**Rationale:** Improperly handled input may enable injection attacks, corrupt stored records, or cause unexpected system behaviour.
**Risk:** High

**SR18:** The system shall terminate or invalidate multiplayer session credentials when a user leaves a session or the session expires.
**Rationale:** Invalidating expired session credentials reduces the risk of unauthorized reuse or continued access to multiplayer sessions.
**Risk:** Medium

**SR7:** The system shall use encrypted communication when transmitting user data or multiplayer session information over a network.
**Rationale:** Unencrypted network traffic may expose user information or session data to unauthorized parties.
**Risk:** High

**SR8:** The system shall restrict access to stored user-specific gameplay records

to the relevant user and authorized system components.
**Rationale:** Unauthorized access may expose private information or allow gameplay records to be modified.
**Risk:** Medium

**SR10:** The system shall preserve previously saved gameplay records when unexpected software errors or network interruptions occur.
**Rationale:** Unexpected failures may otherwise result in corrupted records or the loss of user progress.
**Risk:** Medium

**SR15:** The system shall securely store authentication credentials and shall not store passwords in plaintext. Passwords shall be protected using an appropriate salted password-hashing algorithm.
**Rationale:** Plaintext passwords may be exposed if stored data is accessed by an unauthorized party.
**Risk:** High

**SR17:** The system shall prevent unauthorized modification of stored gameplay records and verify data integrity when records are retrieved or updated.
**Rationale:** Tampered performance records may produce inaccurate results and undermine the reliability of user progress tracking.
**Risk:** Medium

**SR19:** The system shall limit repeated authentication attempts within a defined period.
**Rationale:** Limiting repeated attempts helps reduce automated password-guessing and brute-force attacks.
**Risk:** Medium

**SR20:** The system shall record security-relevant events, including failed authentication attempts and unauthorized access attempts, without unnecessarily logging sensitive user information.
**Rationale:** Security logs support the investigation of suspicious activity while minimizing the exposure of sensitive data.
**Risk:** Medium

# 8 Non-Functional Requirements

**NFR1:** The system shall maintain a frame rate of at least 30 frames per second during standard single-player gameplay on the minimum supported VR hardware.
**Rationale:** Maintaining a consistent frame rate helps reduce visual discomfort and supports an immersive VR experience.
**Risk:** High

**NFR2:** The system shall respond to valid controller inputs within 50 milliseconds under normal operating conditions.
**Rationale:** Low input latency supports precise drone control and provides a more realistic flight simulation experience.
**Risk:** High

**NFR3:** The system shall operate correctly on all VR headsets, controllers, and operating system configurations explicitly listed as supported in the system documentation.
**Rationale:** Defining supported configurations ensures consistent operation and reduces hardware compatibility issues.
**Risk:** Medium

**NFR4:** The system shall restore access to the main menu within 10 seconds of recovering from a recoverable VR tracking or network interruption, provided that the required hardware and services are available.
**Rationale:** Prompt recovery minimizes disruption and reduces the need to restart the application.
**Risk:** Medium

# 9 Roadmap

[Which safety requirements will be implemented as part of the capstone timeline? Which requirements will be implemented in the future? —SS]

# 10 Appendix — Generative AI Use Disclosure

[ChatGPT (version 5) was used to brainstorm ideas for the template for this disclose section. —SS]

## 10.1 AI Tools Used

[Give the name of the AI tool(s) used and the version. —SS]

## 10.2 How and Where Used

[Explain what the tool(s) were used for. Examples include: brainstorming, explaining a technical concept, reviewing the document, generating or modifying text, generating or modifying diagrams, identifying errors or omissions or inconsistencies. —SS]

[For each of the uses, identify which parts of the document were affected. —SS]

[You do not need to report every interaction, but you should disclose the uses that materially contributed to your work —SS]

## 10.3 Verification of AI-Generated Content

[

- Describe how the team checked AI-generated or AI-assisted material before including it in the document.
- In particular, verify factual claims, technical assertions, requirements, calculations, references, and other information for which an AI system may produce incorrect or fabricated results.
- **The team is responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document, regardless of whether material was generated by a human or an AI system.**

—SS]

# 11 Appendix — Reflection

[Not required for CAS 741 —SS]

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?
2. What pain points did you experience during this deliverable, and how did you resolve them?
3. Which of your listed risks had your team thought of before this deliverable, and which did you think of while doing this deliverable? For the latter ones (ones you thought of while doing the Hazard Analysis), how did they come about?
4. Other than the risk of physical harm (some projects may not have any appreciable risks of this form), list at least 2 other types of risk in software products. Why are they important to consider?